

Code Info



Organization
1inch

Repository

token-plugins



Branch
master



Contracts in scope

contracts/ERC20Plugins.sol

Code Statistics

 Findings 5	 Contracts Scanned 1	 Lines of Code 211	 Scan ID 35b24d33-fd55-4bc1-8c82-97bc6121e420
---	--	--	---

Code Summary

The ERC20Plugins contract is an implementation of the ERC20 token standard that allows for the integration and management of external plugins associated with each token holder. The contract defines a maximum limit of plugins per account and regulates the interaction with these plugins to prevent excessive gas consumption. Key functionalities include adding, removing, and counting plugins related to a specific account, as well as ensuring that only compatible plugins (those adhering to the IPlugin interface) can be registered. The contract utilizes a mapping to track the plugins for each account and implements reentrancy safeguards to enhance security during operations. The architecture supports efficient balance management by automatically updating plugin balances during token transfers, allowing for seamless integration of various financial functionalities through plugins. The primary entry points include addPlugin, removePlugin, pluginsCount, and balanceOf, enabling users to manage their plugin interactions while handling ERC20 token balances.

🚩 1 of 5 Findings 📁 ERC20Plugins.sol

Missing revert message on require statements ● Low Risk

Several require statements in the contract lack custom error messages, making debugging difficult when the conditions are not met. For instance, in `_addPlugin``, if the plugin address is found to be zero, reverting without a message provides no context to the caller.

✦ 2 of 5 Findings 📁 ERC20Plugins.sol

Exceeds GAS limit in plugins update ● Medium Risk

The `\_updateBalances` function uses inline assembly to handle plugin updates with a gas limit. If the gas provided is insufficient or there are too many plugins involved in a single token transfer, the function can fail silently, resulting in potential loss of data or unprocessed balances.

✦ 3 of 5 Findings 📁 ERC20Plugins.sol

Potential for token loss due to insufficient checks ● High Risk

In the `addPlugin` function, there are inadequate checks to prevent adding the same plugin multiple times, which could lead to unintended behaviors and make it difficult to manage allowed plugins effectively.

✦ 4 of 5 Findings 📁 ERC20Plugins.sol

Improper handling of token transfers ● High Risk

In the `\_update` function, the logic does not adequately handle cases where the token transfer fails, particularly for external plugins. This can lead to incorrect updates being applied and ultimately to the potential loss of funds.

✦ 5 of 5 Findings 📁 ERC20Plugins.sol

Misleading variable names can cause confusion ● Info

Various variable names and function parameters may not clearly convey their intent or expected data types, leading to confusion for developers interacting with the contract.